3. Testing of the solution

It is vital to check that the algorithm actually solves the problem and produces the expected results for given inputs. In addition to the known results that may have been included in the program specification, other test data should be designed to test the logic of the solution.

A technique for testing the algorithm with test data is the **desk check**. This is a step-by-step walk-through of the logic of the algorithm to see how the input is transformed into output. This is a paper (or whiteboard) exercise that is carried out before the program code is entered on the computer. If the results from the desk check do not match the expected results it is necessary to return to stage 2 to modify the algorithm. The modified algorithm is then retested and modified until the expected results are obtained for the test data. Since the conclusions derived from a desk check are only as good as the test data, it is necessary to design comprehensive test data, which should test all conceivable conditions.

4. Coding the Program

The commands of a programming language are used to transform the algorithm into a computer program after the logic of the program has been planned. Any programming language can be used to code the algorithm, however a particular language may be more suitable than others depending on the type of problem.

During the coding of the program it is good practice to include comments that explain the logic of the processing steps. These comments will make it easier to understand the program at a later date and make the maintenance of the program easier.

A compiler or interpreter is used to convert the program code to machine language that the computer can understand. When the compiler detects an error in the program, an error message is displayed.

There are two basic categories of error messages:

1. **Syntax** and
2. **Run-time**.

The compiler detects and displays syntax errors as it attempts to translate the program. If the program contains a syntax error, it cannot be translated and the program cannot be executed.

Syntax errors occur if the rules of the programming language (the syntax) are not adhered to. This may occur when reserved words (words that have special meaning to that programming language) are misused, misspelled or punctuated incorrectly. These syntax errors must be corrected before the program can be run. The program code is stored in a text file, which can be printed from the program editor, a text editor, or a word processing package. The **program listing** is the documentation for this stage.

Run-time errors are detected and displayed during execution of the program. They occur when the program instructs the computer to perform an illegal operation, such as subscript out of range, dividing a number by zero, type mismatches or manipulating undefined or invalid data. When a run-time error occurs, the computer stops program execution and displays an error message.

5. Testing the Program

This step involves running the program with the same test data used to test the algorithm. If the output does not match the expected results it indicates that errors have been made in converting the algorithm into program code. Debugging the program is the process that identifies these errors.

A number of techniques can be used to check the progress of the processing steps. Many program editors have a **debugger**, which can be used for this purpose. When an error is identified, the program is modified and then tested again. This cycle is repeated until the program gives the required output.

OUTPUT

1. For each order print the details of the order (person/company name, size, density, number of boxes and order cost) to the screen.
2. After the orders have been entered i.e. after the user enters END to finish the session the following are displayed on the screen:
   - The total number of orders made.
   - The total number of boxes of each size (double and high combined).
   - The total cost of all orders.

OVERALL TASKS

1. Initialisations
2. Input and validate order details or END
3. Calculate order cost
4. Update statistics
5. Print details of order
6. Print statistics for all orders entered

Exercise 1

In each of the following write a program specification.

1. **POSTO**: A program outputs to the screen the postage charge for a parcel when the weight of the parcel (in Kg) is entered from the keyboard. The charge is set at $1.50 per kg.

2. **BANKO**: A loan officer requires a program to calculate the monthly repayment M on a customer loan by entering from the keyboard the principal P, at an annual interest rate of R% p.a., over a period of T years where M=P*(1+R*T/100)/(12*T). All inputs and the result is output to the screen.

3. **ANIMO**: The consultation fee for a vet is obtained by multiplying the number of hours by the hourly rate of $25. A program accepts the number of hours (rounded up) from the keyboard and outputs the consultation fee to the screen.

4. **TEMPO**: Convert a temperature entered from the keyboard in degrees Fahrenheit to degrees Celsius where degC=(degF-32)/1.8 and output the result to the screen.

5. **RECTO**: A program accepts the height and width of a rectangle from the keyboard, calculates the perimeter and area of the rectangle and outputs the results to the screen.

6. **DISCO**: A program accepts the marked value of an item in $ and cents, and the discount rate as a % from the keyboard and returns the discounted price to the screen.

7. **TIMEO**: A program allows a person to enter values for the number of days, hours, minutes and seconds and outputs the total number of seconds to the screen.

One means of coming to understand a programming problem is to identify and state the following details:

1. The data provided by the user (INPUT).
2. The processes involved in solving the problem (PROCESSING).
3. The information that is provided to the user (OUTPUT).

A program will usually require some form of **input**, be it information to process or the selection of a processing option from a menu. You usually cannot do any processing until you have completed the input of information to process.

The program will often then **process** some of the input and convert it to some form of information based on the data supplied. The processing steps should be described only as actions at this stage. The details about how each step will be carried out are developed as part of the algorithm design.

The last section of a program often involves **output**. This can be in the form of a report displayed on the screen or output to a printer, or it could be information written to some sort of storage device, such as disk or tape.

The IPO chart should be able to be read and understood by a non-programmer. It may provide the basis for discussion with the person who proposed the program if ambiguous details need to be clarified. Thus, the entries in the IPO chart should be in plain English.

Example 1.2

Problem: A program is required that will allow a user to enter a temperature in degrees Celsius and will display the temperature in degrees Fahrenheit.

Problem definition:
An analysis of the problem indicates a single input and a single output. The input has to be converted to the output.

IPO CHART

INPUT	PROCESSING	OUTPUT
Temperature in degrees Celsius	1. Enter the temperature in degrees Celsius	Temperature in degrees Fahrenheit
	2. Convert the temperature in degrees Celsius to the temperature in degrees Fahrenheit	
	3. Display the temperature in degrees Fahrenheit	